

OS LAB WEEK 8

Implementation of Worst Fit, Best Fit, First Fit

```
#include <stdio.h>

#define MAX 25

void firstFit(int blockSizes[], int n, int processSizes[], int m) {
    int allocation[MAX];

    for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {

            // Allocate only once
            if (blockSizes[j] >= processSizes[i]) {
                allocation[i] = j + 1;

                // Mark block as used
                blockSizes[j] = 0;
                break;
            }
        }
    }

    printf("\n--- First Fit ---\n");
    printf("%-15s %-15s %s\n", "Process No.", "Process Size", "Block No.");

    for (int i = 0; i < m; i++) {
        printf("%-15d %-15d", i + 1, processSizes[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSizes[], int n, int processSizes[], int m) {
    int allocation[MAX];

    for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++) {
```

```

int bestBlock = -1;

for (int j = 0; j < n; j++) {

    if (blockSizes[j] >= processSizes[i]) {

        if (bestBlock == -1 || blockSizes[j] < blockSizes[bestBlock])
            bestBlock = j;
    }
}

if (bestBlock != -1) {
    allocation[i] = bestBlock + 1;

    // Mark block as used
    blockSizes[bestBlock] = 0;
}
}

printf("\n--- Best Fit ---\n");
printf("%-15s %-15s %s\n", "Process No.", "Process Size", "Block No.");

for (int i = 0; i < m; i++) {
    printf("%-15d %-15d", i + 1, processSizes[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i]);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blockSizes[], int n, int processSizes[], int m) {
    int allocation[MAX];

    for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++) {
        int worstBlock = -1;

        for (int j = 0; j < n; j++) {

            if (blockSizes[j] >= processSizes[i]) {

                if (worstBlock == -1 || blockSizes[j] > blockSizes[worstBlock])
                    worstBlock = j;
            }
        }
    }
}

```

```

        if (worstBlock != -1) {
            allocation[i] = worstBlock + 1;

            // Mark block as used
            blockSizes[worstBlock] = 0;
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("%-15s %-15s %s\n", "Process No.", "Process Size", "Block No.");

    for (int i = 0; i < m; i++) {
        printf("%-15d %-15d", i + 1, processSizes[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    int n, m;

    int blockSizes1[MAX], blockSizes2[MAX], blockSizes3[MAX];
    int originalBlocks[MAX];
    int processSizes[MAX];

    printf("Enter number of memory blocks: ");
    scanf("%d", &n);

    printf("Enter sizes of %d memory blocks:\n", n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &originalBlocks[i]);

        blockSizes1[i] = originalBlocks[i];
        blockSizes2[i] = originalBlocks[i];
        blockSizes3[i] = originalBlocks[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &m);

    printf("Enter sizes of %d processes:\n", m);

    for (int i = 0; i < m; i++)
        scanf("%d", &processSizes[i]);

    firstFit(blockSizes1, n, processSizes, m);

```

```
bestFit(blockSizes2, n, processSizes, m);  
worstFit(blockSizes3, n, processSizes, m);  
  
return 0;  
}
```

Enter number of memory blocks: 5

Enter sizes of 5 memory blocks:

100 500 200 300 600

Enter number of processes: 4

Enter sizes of 4 processes:

212 417 112 426

--- First Fit ---

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

--- Best Fit ---

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	4